

Retos: patrones restantes

Compilación breve de patrones no cubiertos en el resto de transparencias.

Fuentes principales (Refactoring.Guru):

Builder — Propósito

Patrón creacional.

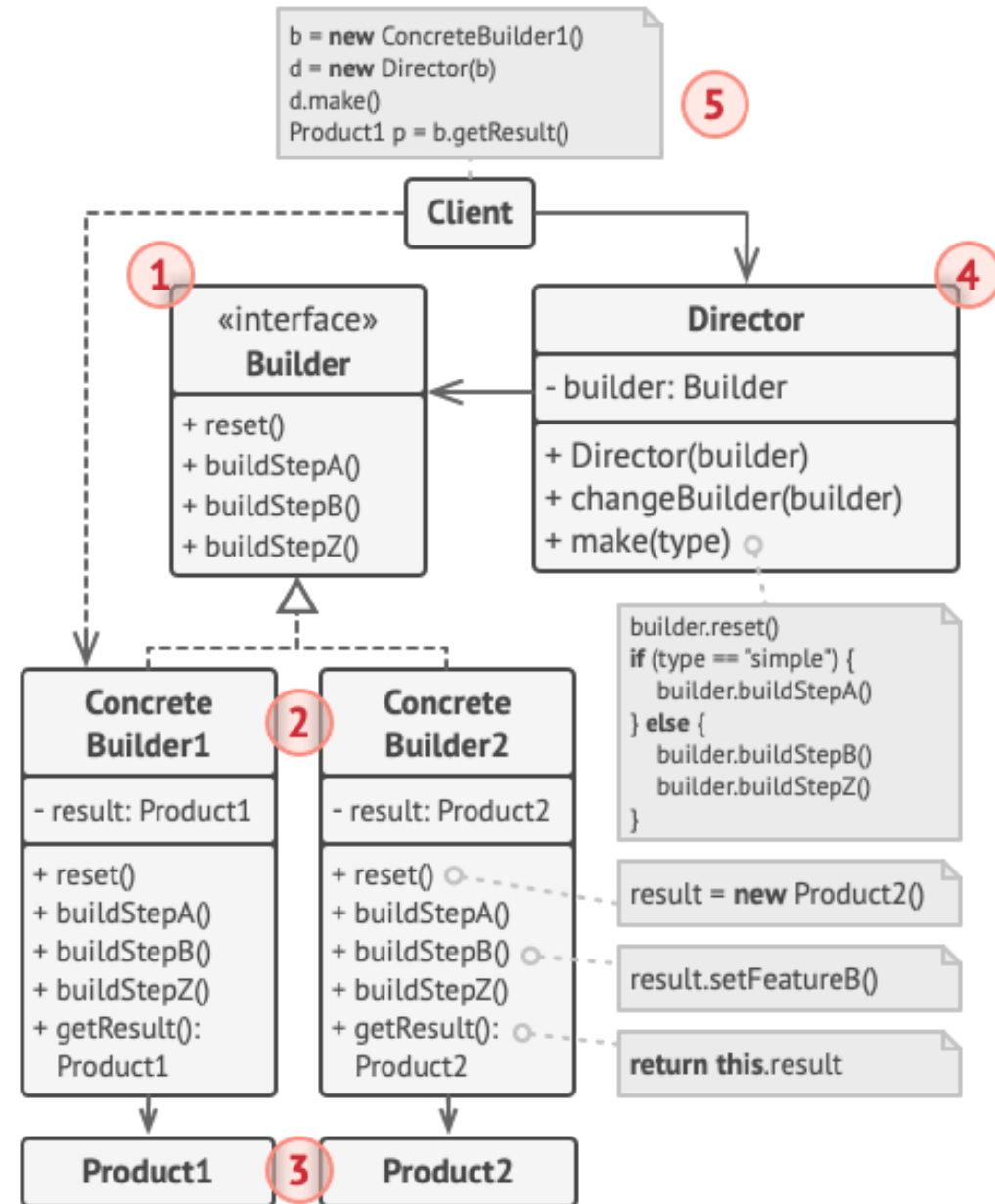
- Construir objetos complejos **paso a paso**.
- Reutilizar el mismo proceso de construcción para crear **distintas representaciones**.

Builder — Problema

- Constructores “telescópicos” con muchos parámetros opcionales.
- Explosión de subclases para cubrir combinaciones.
- Código de construcción difícil de leer, probar y mantener.

Builder — Solución

- Extraer la construcción a un **Builder** con pasos (p. ej. `setX`, `setY`, ...).
- Opcionalmente, un **Director** define el orden de pasos.
- Permite productos/variantes sin inflar constructores ni jerarquías.



Prototype — Propósito

Patrón creacional.

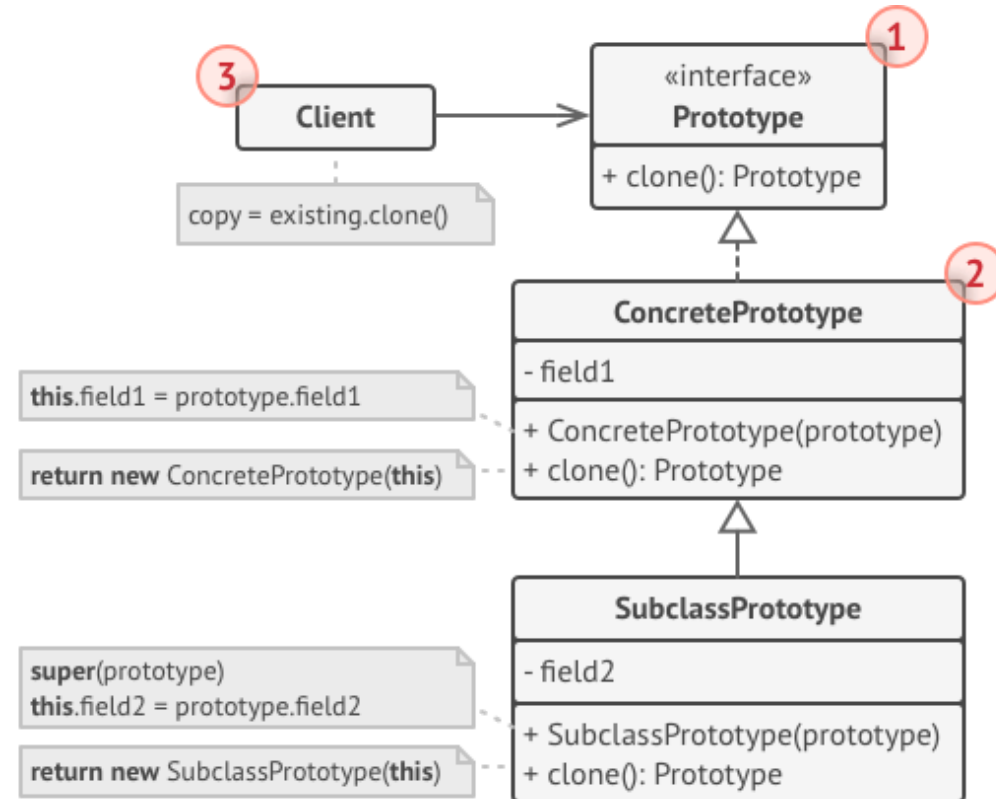
- Crear copias de objetos existentes **sin acoplarte** a sus clases concretas.
- Útil cuando instanciar/configurar desde cero es costoso.

Prototype — Problema

- Copiar “desde fuera” puede ser imposible (campos privados, referencias, etc.).
- El código cliente se acopla a clases concretas para duplicar instancias.

Prototype — Solución

- Definir una interfaz común de clonación (p. ej. `clonar()`).
- Cada clase implementa su copia (incluyendo casos especiales si los hay).
- Registro de prototipos opcional para clonar “plantillas” configuradas.



Adapter — Propósito

Patrón **estructural**.

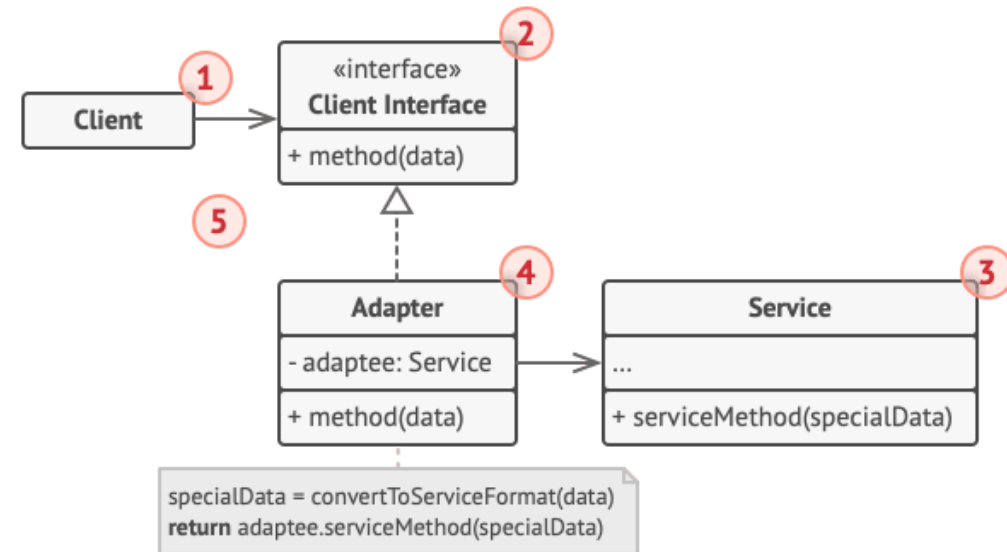
- Permitir que colaboren objetos con interfaces **incompatibles**.
- Hacer reutilizable un servicio/librería sin modificarla.

Adapter — Problema

- Tu código espera una interfaz/formato, pero una librería externa ofrece otro.
- Cambiar la librería o tu código puede ser inviable o arriesgado.

Adapter — Solución

- Crear un **adaptador** que implemente la interfaz esperada por el cliente.
- El adaptador **envuelve** el servicio real y traduce llamadas/datos.



Bridge — Propósito

Patrón **estructural**.

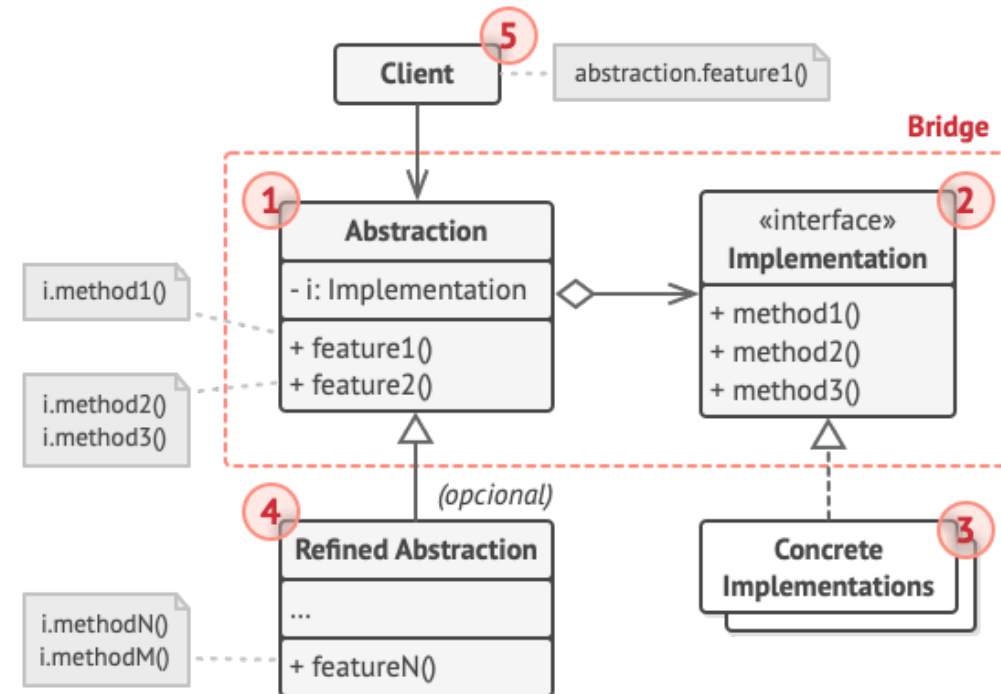
- Separar una abstracción de su implementación.
- Poder evolucionar ambas jerarquías de forma **independiente**.

Bridge — Problema

- Una jerarquía crece sin control al combinar 2 dimensiones (p. ej. forma × color).
- Añadir una variante en una dimensión obliga a multiplicar subclases.

Bridge — Solución

- Sustituir herencia por **composición**: la abstracción contiene una implementación.
- Cambiar/añadir implementaciones sin tocar la jerarquía de abstracciones (y viceversa).



Facade — Propósito

Patrón **estructural**.

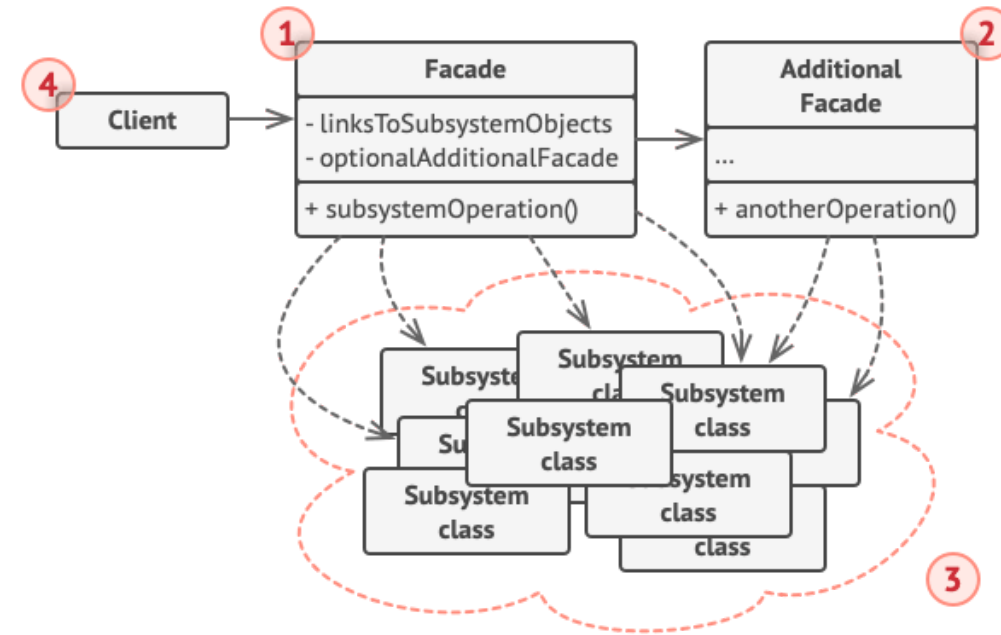
- Ofrecer una interfaz **simple** a un subsistema complejo.
- Reducir acoplamiento y simplificar el uso desde el cliente.

Facade — Problema

- Para usar un subsistema hay que coordinar muchos objetos, dependencias y orden.
- La lógica de negocio acaba mezclada con detalles de integración.

Facade — Solución

- Crear una **fachada** con métodos de alto nivel ("casos típicos").
- La fachada conoce el subsistema y delega/coordina por dentro.



Flyweight — Propósito

Patrón **estructural**.

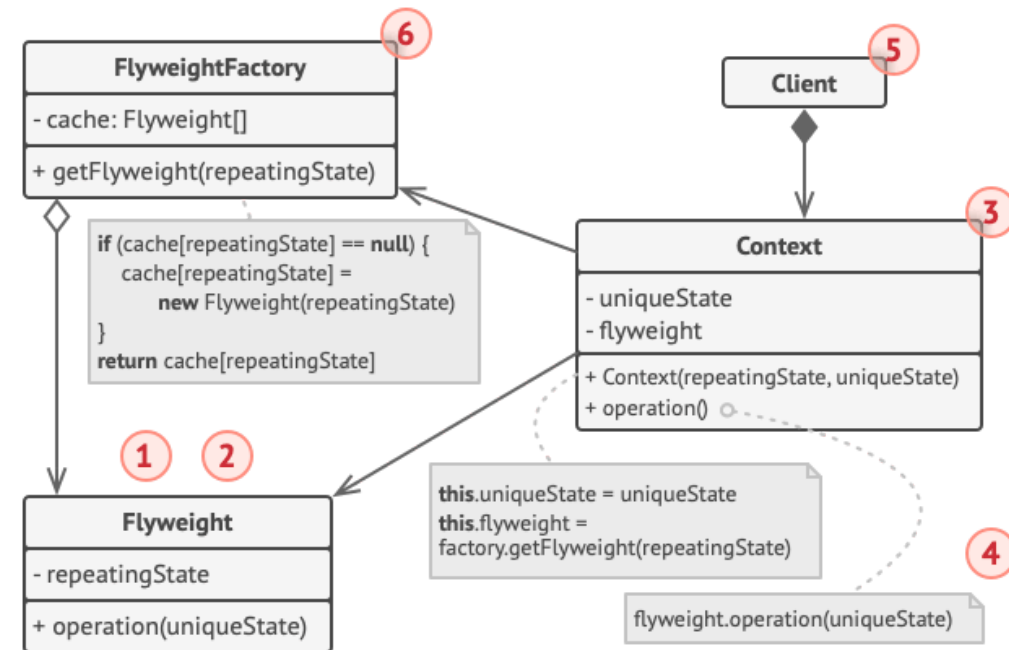
- Ahorrar memoria cuando hay **muchísimos** objetos similares.
- Compartir la parte común del estado entre instancias.

Flyweight — Problema

- Cada objeto guarda datos repetidos (texturas, modelos, estilos, etc.).
- El número de instancias hace que la app se quede sin RAM.

Flyweight — Solución

- Separar estado **intrínseco** (compartido) y **extrínseco** (contexto).
- Reutilizar flyweights mediante una **fábrica/caché**.



Proxy — Propósito

Patrón **estructural**.

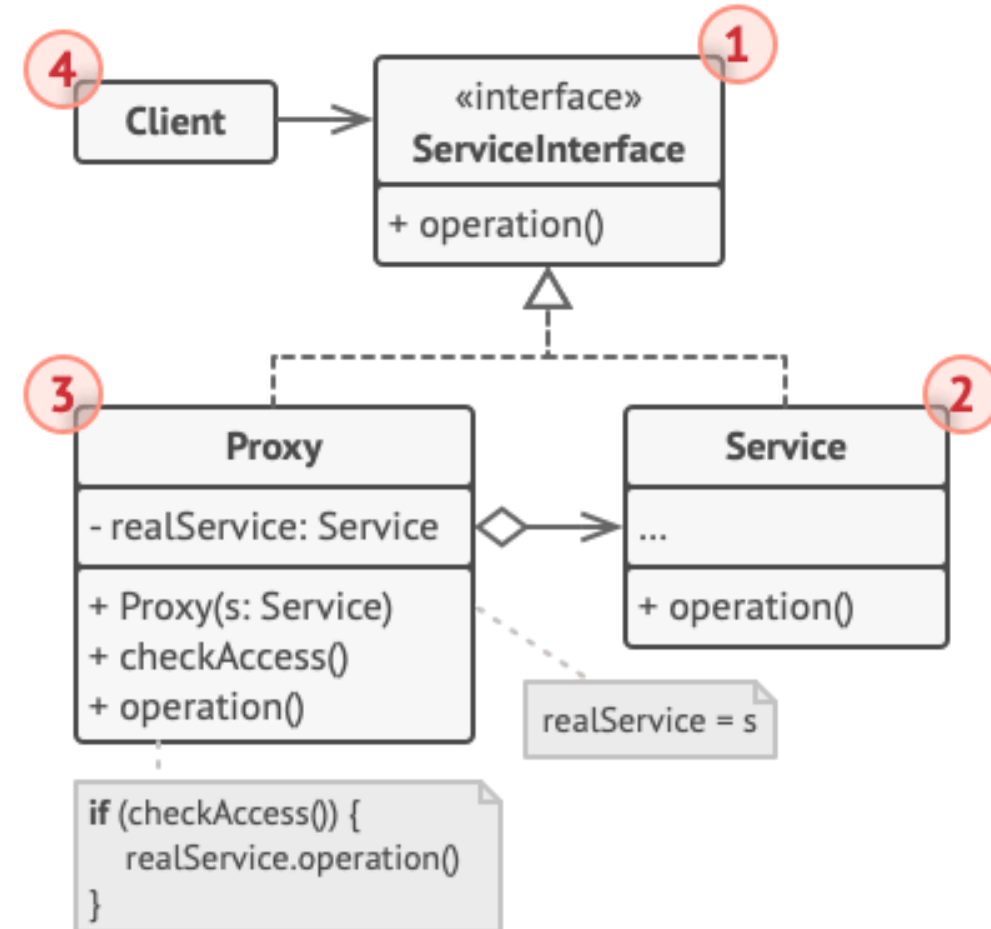
- Proveer un sustituto que controla el acceso a un objeto real.
- Añadir comportamiento antes/después sin cambiar el objeto real.

Proxy — Problema

- Objetos “pesados” o remotos: inicialización lenta, permisos, caché, logging...
- Meter esa lógica en el cliente duplica código y acopla.

Proxy — Solución

- El proxy expone la misma interfaz que el servicio.
- Intercepta llamadas (p. ej. lazy init / control de acceso / caché) y delega.



Chain of Responsibility — Propósito

Patrón de comportamiento.

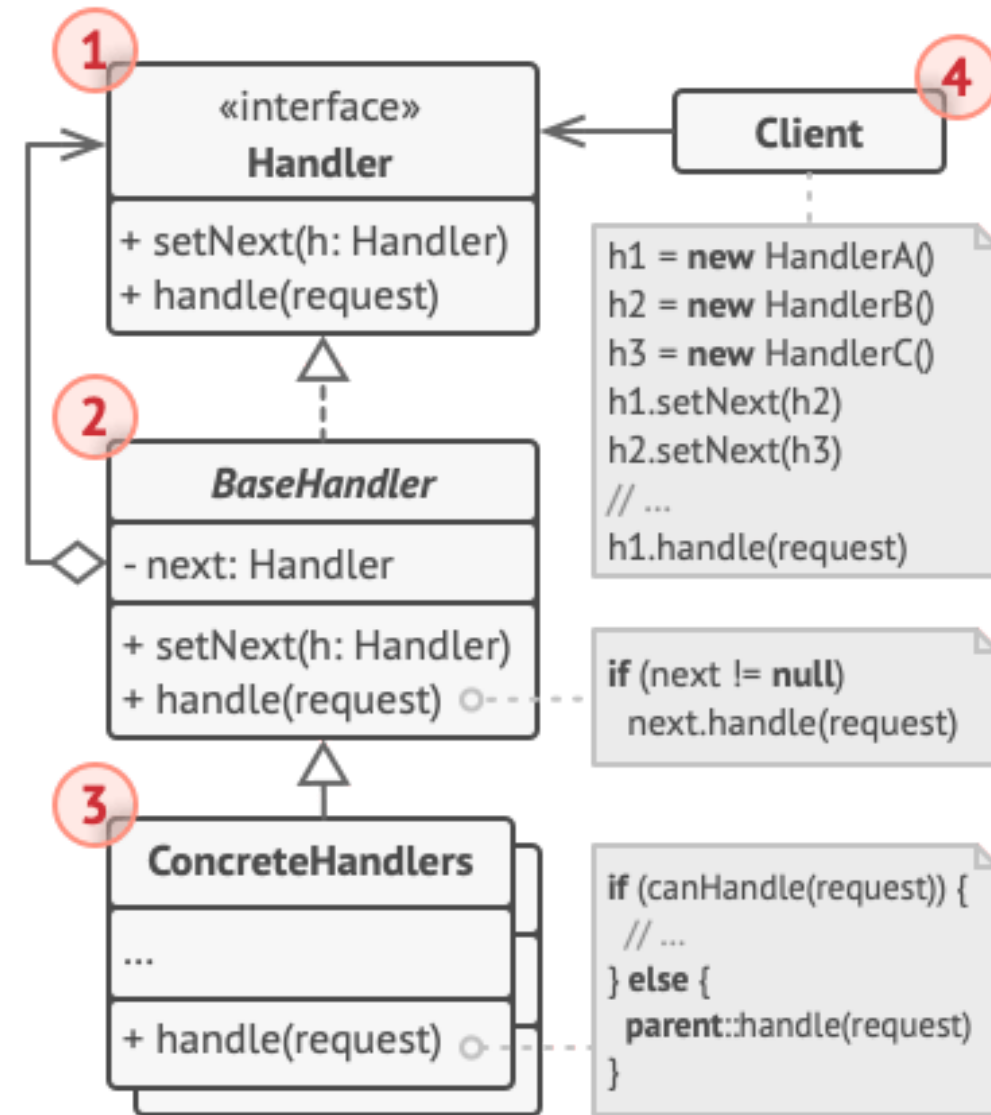
- Pasar una solicitud por una **cadena** de manejadores.
- Cada manejador decide si procesa o delega al siguiente.

Chain of Responsibility — Problema

- Secuencias de comprobaciones/acciones con `if` anidados y dependencias.
- Reutilizar una parte del flujo obliga a copiar y pegar.

Chain of Responsibility — Solución

- Convertir cada paso en un **handler** con referencia a `next`.
- Componer la cadena en runtime y cambiar el orden sin tocar el cliente.



Command — Propósito

Patrón de comportamiento.

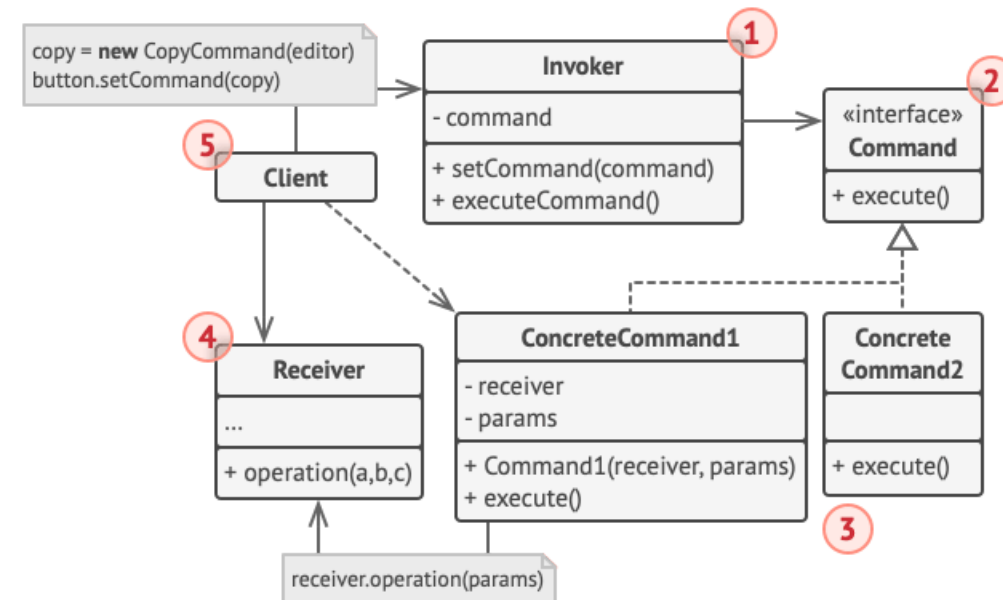
- Encapsular una operación como un objeto (un “comando”).
- Facilitar colas, logs, macros y **deshacer/rehacer**.

Command — Problema

- UI y lógica se acoplan: botones/menús/atajos duplican manejadores.
- Implementar historial y deshacer es complejo si todo son llamadas directas.

Command — Solución

- Definir interfaz `execute()` (y opcional `undo()`).
- El **invocador** dispara comandos; el **receptor** hace el trabajo real.



Iterator — Propósito

Patrón de comportamiento.

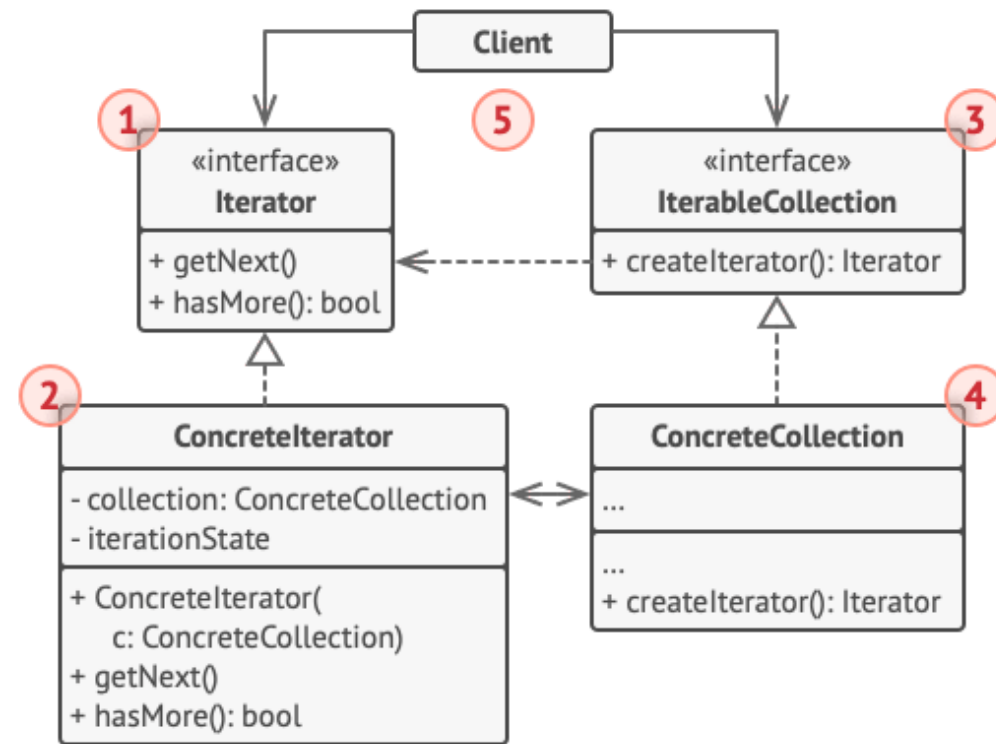
- Recorrer colecciones sin exponer su representación interna.
- Permitir múltiples recorridos (y varios iteradores a la vez).

Iterator — Problema

- Si metes recorridos en la colección: la clase crece y mezcla responsabilidades.
- Si el cliente recorre “a su manera”: se acopla a cada estructura interna.

Iterator — Solución

- Extraer el recorrido a un **iterador** con estado (posición, siguiente...).
- Colecciones exponen un método para crear iteradores compatibles.



Memento — Propósito

Patrón de comportamiento.

- Guardar/restaurar el estado previo de un objeto sin violar encapsulación.
- Base típica para `undo`.

Memento — Problema

- Hacer snapshots desde fuera obliga a acceder a detalles internos.
- Cualquier cambio interno rompe el código que serializa/restaura.

Memento — Solución

- El **originador** crea un memento con su estado.
- El **cuidador** guarda el historial y pide restaurar cuando toca.

